Project – Low Level Design

On

Multimodal Manufacturing Creator

Course Name: Data Analytics

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	AADITI BHALE	EN22CS303001
2.	ABHIGYAN SINGH THAKUR	EN22CS303004
3.	ADITYA PUROHIT	EN22CS303007
4.	DHRUV JAIN	EN22CS304022
5.	DIVYANSH TYAGI	EN22CS304024

Group Name: 06D11

Project Number: GAI-32

Industry Mentor Name: AASHRUTI SHAH

University Mentor Name: HEMLATA PATEL

Academic Year: 2022-2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. References

1.Introduction

The Multimodal Manufacturing Creator is a Generative AI-based system designed to convert user-provided manufacturing prompts into detailed textual descriptions and visual prototypes.

This system integrates:

- Large Language Models (LLMs) for generating explanations
- Image generation models for producing design visuals
- Flask backend for API handling
- Docker and Kubernetes for scalable deployment

The system demonstrates how AI can assist in automating manufacturing concept design, making it faster and more efficient.

1.1 Scope of the Document

This document provides a detailed low-level design of the system, including:

- System architecture and workflow
- Backend and frontend design
- API structure
- Integration of LLM and image generation
- Deployment strategy

It focuses on technical implementation details required to build and deploy the system

1.2 Intended Audience

This document is intended for:

- Developers implementing the system
- Project reviewers and faculty

- DevOps engineers
- AI/ML engineers
- Students learning Generative AI systems

1.3 System Overview

The system allows users to:

1. Enter a manufacturing-related prompt
2. Generate:
    - Textual explanation using LLM
    - Visual prototype using image generation
3. View results through a web interface

The application is deployed using:

- Docker containers
- Kubernetes cluster (Minikube)

ensuring scalability, reliability, and fault tolerance.

2. System Design

2.1 Application Design

The system follows a modular architecture consisting of:

Layers:

1. Presentation Layer (Frontend)
    - Handles user interaction
    - Displays output
2. Application Layer (Backend)
    - Processes user requests
    - Communicates with AI services
3. AI Services Layer
    - LLM API (text generation)
    - Image generation API
4. Deployment Layer
    - Docker container
    - Kubernetes orchestration

Design Approach:

- REST API-based communication
- Microservices-like modular design
- Scalable and loosely coupled components

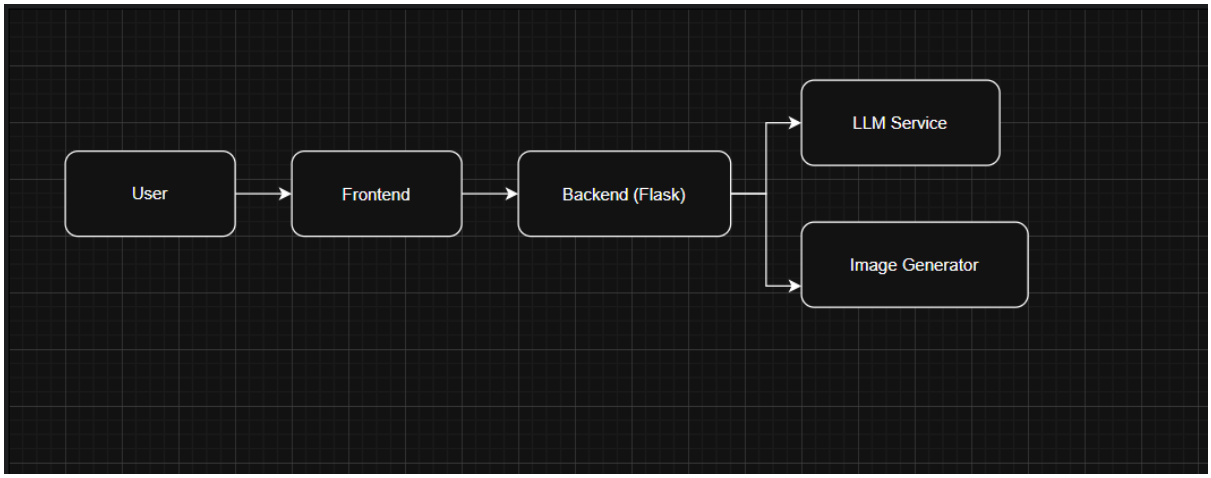
2.2 Process Flow

The process flow describes how a request moves through the system.

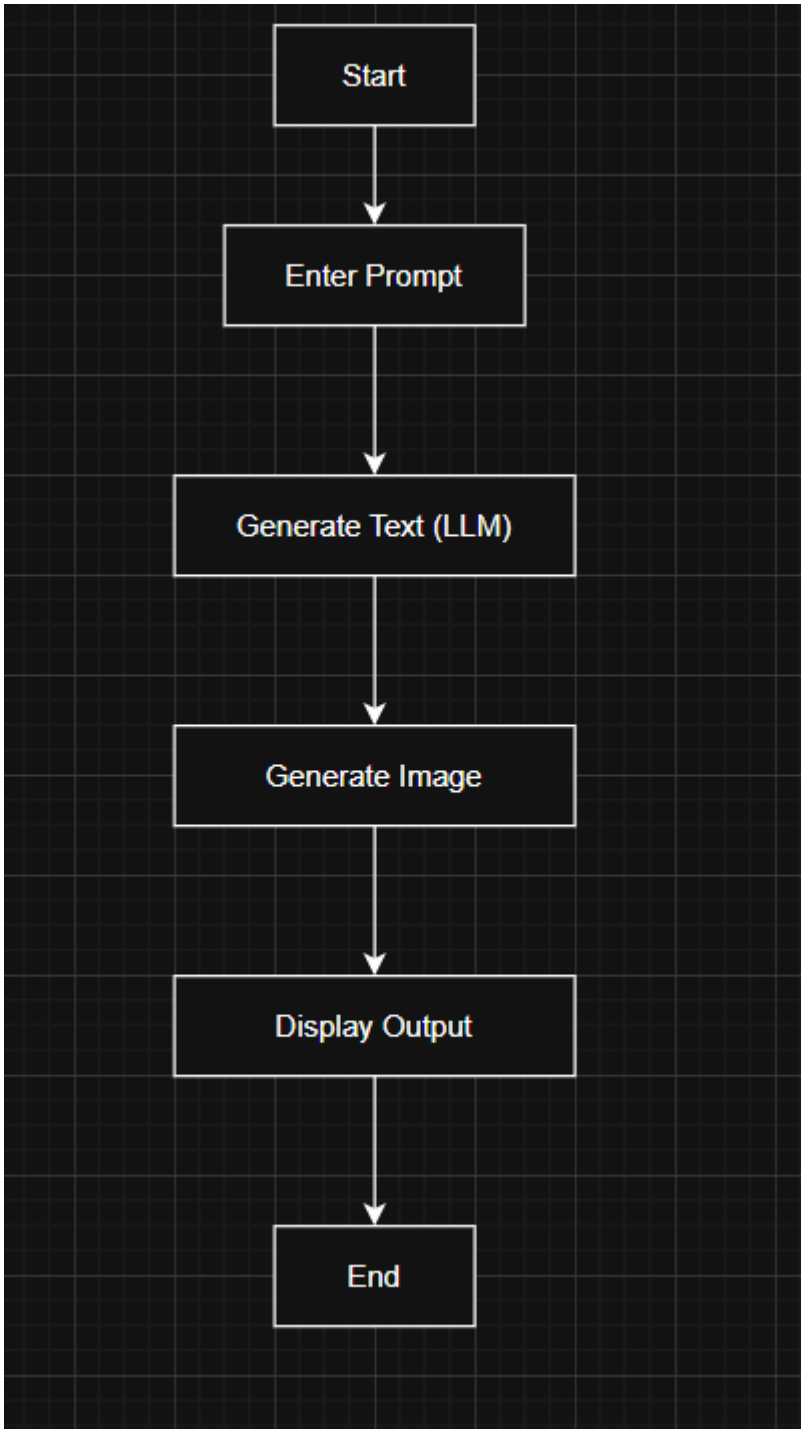
Steps:

1. User enters prompt
2. Frontend sends request to backend
3. Backend receives and validates input
4. Backend calls LLM API
5. Backend calls image generation API
6. Backend combines responses
7. Response sent to frontend
8. Output displayed to user

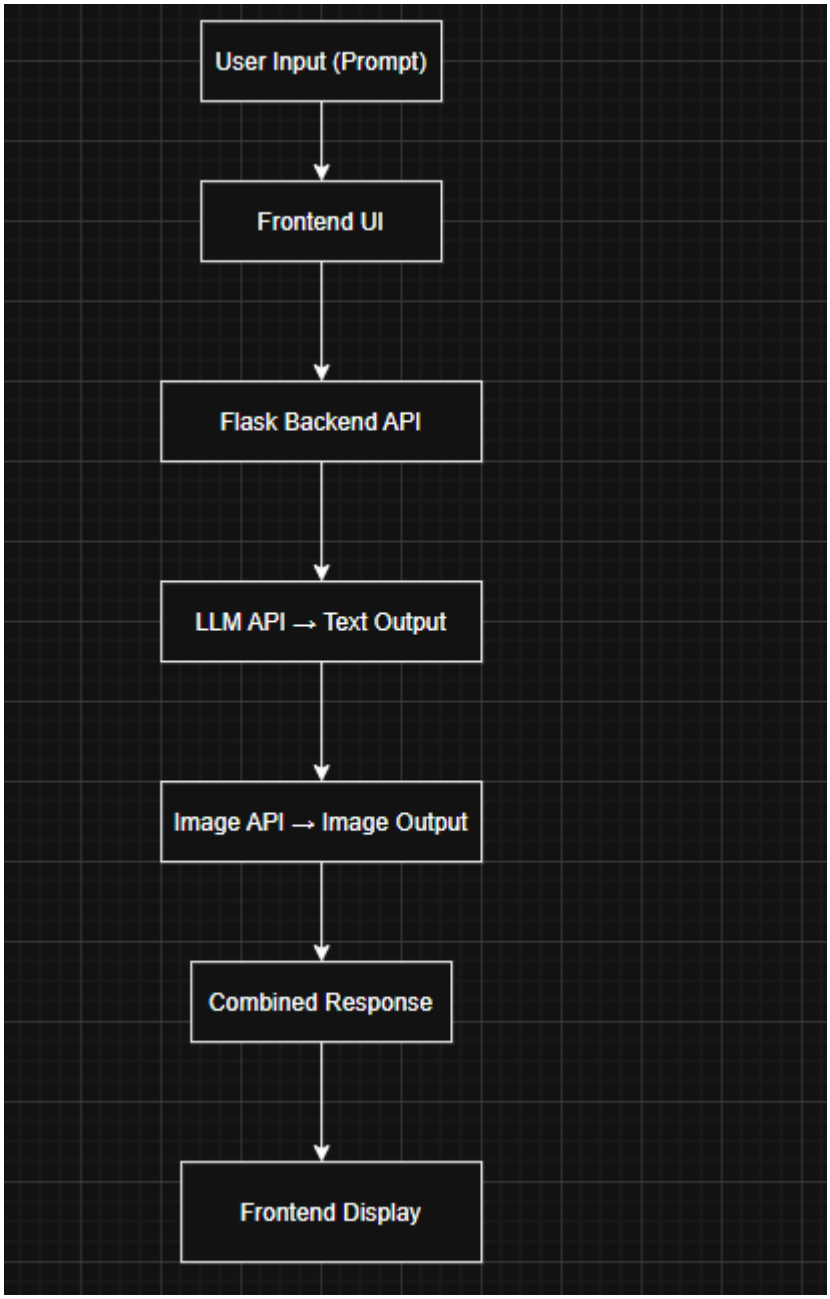
Class Diagram:



Activity Diagram:



2.3 Information Flow



Data Types:

- Input: Text prompt
- Output:
  - JSON response
  - Text description
  - Image URL

2.4 Components Design

The system consists of the following components:

1. User

- Provides input prompt
- Views generated output

2. Frontend

- Built using HTML + JavaScript
- Sends API requests
- Displays results

3. Backend (Flask API)

- Core processing unit
- Handles API requests
- Integrates AI services

4. LLM Service

- Generates textual explanation
- Processes natural language input

5. Image Generation Service

- Creates visual representation
- Uses diffusion-based models

6. Docker Container

- Packages application and dependencies
- Ensures portability

7. Kubernetes Cluster

- Manages deployment
- Handles scaling and self-healing

2.5 Key Design Considerations

1. Scalability

- Kubernetes ensures horizontal scaling
- Supports multiple users

2. Availability

- Multiple replicas ensure uptime
- Self-healing pods

3. Performance

- Fast LLM (Groq API)
- Efficient API calls

4. Modularity

- Separate frontend, backend, AI services

5. Security

- API keys managed via environment variables
- Secure communication (future enhancement: HTTPS)

6. Maintainability

- Clean code structure
- Modular design

2.6 API Catalogue

1. Generate Design API

Endpoint:

```bash
POST /generate-design
```

Description:
Generates manufacturing concept and image.

Request Body:

```json
{
  "prompt": "robotic arm for assembly"
}
```

Response:

```json
{
  "prompt": "robotic arm",
  "description": "AI generated explanation",
  "image": "image_url"
}
```

2. Health API

```bash
GET /health
```

Purpose:
Check system health

Response:

OK

3. Status API

```bash
GET /status
```

Response:

```json
{
  "machine": "operational",
  "version": "v3"
}
```

3. References

1. Flask Documentation,
Available at: https://flask.palletsprojects.com/
(Used for backend development)

2. Docker Documentation,
Available at: https://docs.docker.com/
(Used for containerization)

3. Kubernetes Documentation,
Available at: https://kubernetes.io/docs/
(Used for deployment and orchestration)

4. Groq API Documentation,
Available at: https://console.groq.com/docs
(Used for LLM integration)

5. Replicate API Documentation,
Available at: https://replicate.com/docs
(Used for image generation)

6. HTML & JavaScript Documentation (MDN Web Docs),
Available at: https://developer.mozilla.org/
(Used for frontend development)

7. Generative AI Concepts,
Available at: https://www.ibm.com/topics/generative-ai
(Basic understanding of GenAI systems)